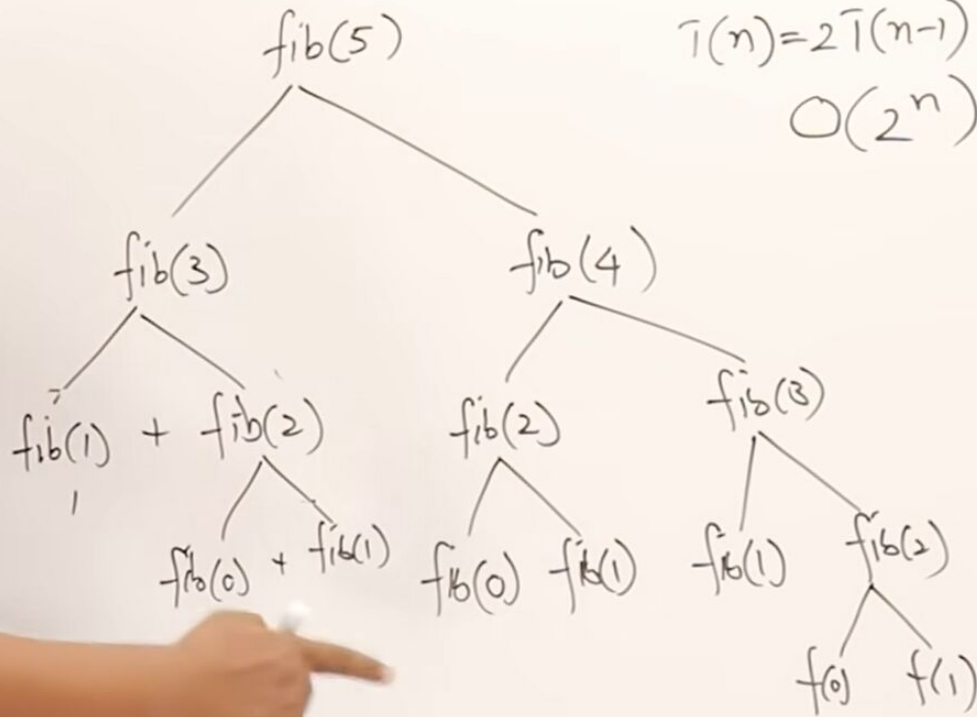


Dynamic Programming

0, 1, 1, 2, 3, 5, 8, 13
0 1 2 3 4 5 6 7 - - -



$$T(n) = 2T(n-1) + 1$$
$$O(2^n)$$

$$\text{fib}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{fib}(n-2) + \text{fib}(n-1) & \text{if } n > 1 \end{cases}$$

```
{ int fib(int n)
  if (n <= 1)
    return n;
  return fib(n-2) + fib(n-1);
}
```



Dynamic Programming

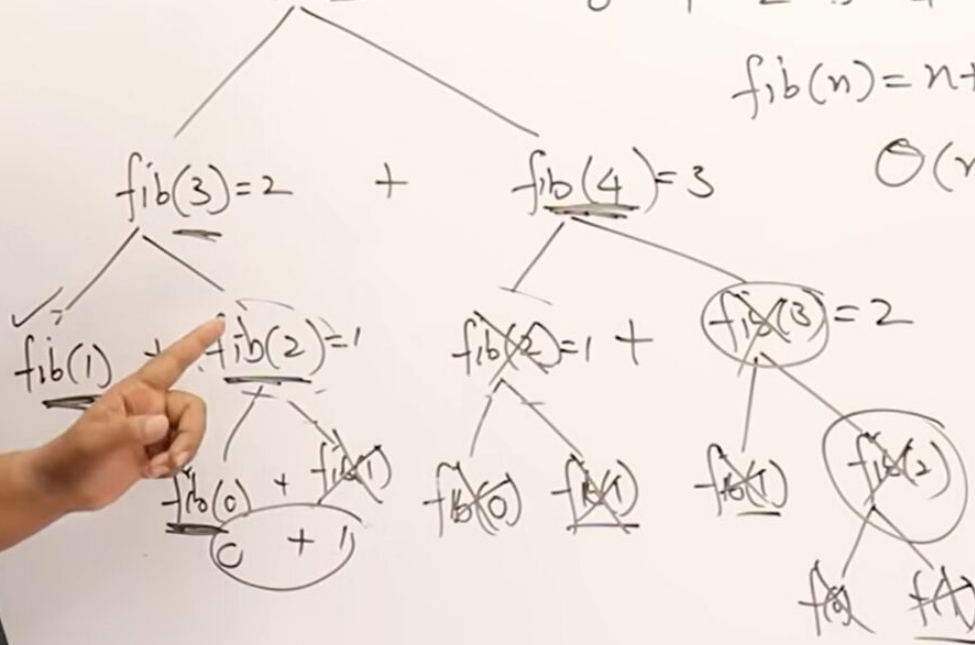
0, 1, 1, 2, 3, 5, 8, 13
0 1 2 3 4 5 6 7 - - -

$\text{fib}(5) = 5$ F

0	1	1	2	3	5
0	1	2	3	4	5

$\text{fib}(n) = n+1$ calls

$O(n)$



$$\text{fib}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{fib}(n-2) + \text{fib}(n-1) & \text{if } n > 1 \end{cases}$$

```
int fib(int n)
{
    if (n <= 1)
        return n;
    return fib(n-2) + fib(n-1);
}
```

are avoiding the same call once again
like fib



Dynamic Programming

F	0	1	1	2	3	5
	0	1	2	3	4	5

fib(5)

$$\text{fib}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{fib}(n-2) + \text{fib}(n-1) & \text{if } n > 1 \end{cases}$$

```
int fib(int n)
{
    if (n <= 1)
        return n;
    F[0] = 0; F[1] = 1;
    for (int i = 2; i <= n; i++)
    {
        F[i] = F[i-2] + F[i-1];
    }
    return F[n];
}
```

Dynamic Programming

F

0	1	1	2	3	5
0	1	2	3	4	5

fib(5)

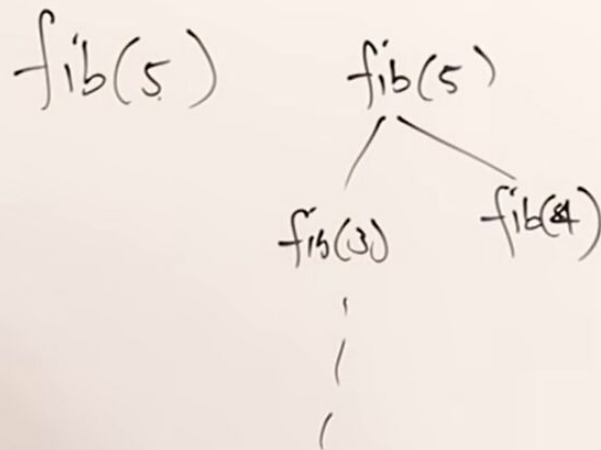
seized from smaller value onwards so we say this is bottom-up

$$\text{fib}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{fib}(n-2) + \text{fib}(n-1) & \text{if } n > 1 \end{cases}$$

```
int fib(int n)
{
    if (n <= 1)
        return n;
    F[0] = 0; F[1] = 1;
    for (int i = 2; i <= n; i++)
    {
        F[i] = F[i-2] + F[i-1];
    }
    return F[n];
}
```

Dynamic Programming

0	1	1	2	3	5
0	1	2	3	4	5



going till bottom then it was coming back again so it was a top-down but

$$\text{fib}(n) = \begin{cases} 0 & \text{if } n=0 \\ 1 & \text{if } n=1 \\ \text{fib}(n-2) + \text{fib}(n-1) & \text{if } n > 1 \end{cases}$$

```
int fib(int n)
{
    if (n <= 1)
        return n;
    F[0] = 0; F[1] = 1;
    for (int i = 2; i <= n; i++)
    {
        F[i] = F[i-2] + F[i-1];
    }
    return F[n];
}
```

